

Optimization Algorithms for Machine Learning

Instructor:

Objective

To understand the intuition, mechanics, and differences between Gradient Descent, Newton's Method, and their stochastic variants.

1 The Core Problem: Minimizing a Loss Function

In machine learning, we aim to find the parameters (often denoted as θ or $\mathbf{w}$) of a model that minimize a **loss function** $J(\theta)$. This function measures how wrong the model's predictions are compared to the true data.

- $J(\theta)$ = Average loss over the entire training dataset.
- For a dataset with m examples, $J(\theta) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$, where L is a per-example loss (e.g., squared error).

2 Gradient Descent (GD) – “The Batch Method”

Intuition: Imagine you're at the top of a foggy mountain (high loss) and want to get to the bottom (minimum loss). The fastest way down is to always take a step in the direction of the steepest slope. The gradient (∇J) is a vector that points *uphill* in the direction of steepest ascent. Therefore, $-\nabla J$ points *downhill*.

Algorithm:

1. Initialize parameters θ (e.g., randomly or to zero).
2. **Repeat until convergence:**
 - **Compute the gradient** of the cost function with respect to θ for the **entire training dataset**: $\mathbf{g} = \nabla J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$
 - **Update the parameters**: $\theta = \theta - \eta \cdot \mathbf{g}$

where η (eta) is the **learning rate**, a hyperparameter controlling the step size.

Pros: Direct path towards the minimum. Stable convergence.

Cons: **Extremely computationally expensive** for large datasets (m is large). Each step requires a full pass through the entire dataset.

3 Stochastic & Mini-Batch Gradient Descent

To solve the computational problem of Batch GD, we use approximations of the true gradient.

3.1 Stochastic Gradient Descent (SGD)

Intuition: Instead of asking everyone on the mountain for the average direction of steepest descent (the true gradient), you ask just **one randomly chosen person**. Their direction is noisy and inaccurate but points, on average, in the right direction.

Algorithm:

1. Initialize parameters θ .
2. **Shuffle** the training dataset.
3. **For each example** i in the dataset:
 - **Compute the gradient** for that **single example** i : $\mathbf{g}^{(i)} = \nabla L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$
 - **Update the parameters**: $\theta = \theta - \eta \cdot \mathbf{g}^{(i)}$

Pros: Very fast per-step computation. Can escape shallow local minima due to noise.

Cons: Noisy updates lead to a chaotic convergence path. May never exactly converge to the minimum; it will oscillate around it.

3.2 Mini-Batch Gradient Descent

Intuition: A practical compromise. You ask a **small group (a mini-batch)** of people for the average direction. It's less noisy than one person but much faster than asking everyone.

Algorithm:

1. Initialize parameters θ .
2. **Shuffle** the training dataset.
3. **For each mini-batch** B of size b (e.g., 32, 64, 128):
 - **Compute the gradient** for the **mini-batch**: $\mathbf{g}_B = \frac{1}{b} \sum_{i \in B} \nabla L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$
 - **Update the parameters**: $\theta = \theta - \eta \cdot \mathbf{g}_B$

Pros:

- Smoother convergence than SGD (reduced noise).
- Significant speedup over Batch GD, especially on parallelizable hardware (GPUs).
- This is the **de facto standard** for training most deep learning models.

4 Newton's Method (A 2nd-Order Method)

Intuition: Gradient descent uses only first-order information (the slope). Newton's Method also uses second-order information (the **curvature**). It doesn't just ask "which way is down?" but also "how curved is the path? Should I take a bigger or smaller step?"

It uses the **Hessian matrix (H)**, which contains the second derivatives of J with respect to the parameters. The Hessian describes the local curvature of the loss function.

Algorithm:

1. Initialize parameters θ .
2. **Repeat until convergence:**
 - Compute the gradient $\mathbf{g} = \nabla J(\theta)$
 - Compute the Hessian matrix $\mathbf{H} = \nabla^2 J(\theta)$
 - **Update the parameters:** $\theta = \theta - \mathbf{H}^{-1}\mathbf{g}$

Why $\mathbf{H}^{-1}\mathbf{g}$? The term $\mathbf{H}^{-1}$ effectively acts as an adaptive, parameter-specific learning rate. It causes the algorithm to take larger steps in flat directions (low curvature) and smaller steps in steep directions (high curvature).

Pros: Much faster convergence (fewer iterations) near a minimum. No learning rate to tune.

Cons: **Extremely expensive** to compute and invert the Hessian for high-dimensional parameters. Can diverge if $\mathbf{H}$ is not positive definite.

5 Example with Two Variables

Let's define a simple quadratic loss function with two parameters, θ_0 and θ_1 .

Loss Function: $J(\theta) = (\theta_0 - 2)^2 + 2(\theta_1 - 3)^2 + 5$

The constant +5 doesn't affect the optimization. The minimum is clearly at $\theta^* = (2, 3)$.

Gradients:

$$\begin{aligned}\frac{\partial J}{\partial \theta_0} &= 2(\theta_0 - 2) \\ \frac{\partial J}{\partial \theta_1} &= 4(\theta_1 - 3)\end{aligned}$$

Hessian Matrix (for Newton's Method):

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 J}{\partial \theta_0^2} & \frac{\partial^2 J}{\partial \theta_0 \partial \theta_1} \\ \frac{\partial^2 J}{\partial \theta_1 \partial \theta_0} & \frac{\partial^2 J}{\partial \theta_1^2} \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}, \quad \mathbf{H}^{-1} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{4} \end{bmatrix}$$

Initialization: $\theta^{(0)} = (\theta_0, \theta_1) = (0, 0)$.

Learning Rate (for GD variants): $\eta = 0.1$.

Our Toy Dataset:

We will create a tiny dataset of 4 examples that represents our loss function.

- *Example 1:* Contributes to the $(\theta_0 - 2)^2$ part of the loss.
- *Example 2, 3, 4:* Contribute to the $(\theta_1 - 3)^2$ part of the loss.

This is a pedagogical construct to make the mini-batch example clear.

Calculations for Two Steps

A. Batch Gradient Descent

- **Step 0:** $\theta_0 = 0, \theta_1 = 0$
- **Step 1:**
 - Compute gradient on **all 4 examples**:

$$g_0 = \frac{1}{4} [2(0 - 2) + 0 + 0 + 0] = \frac{1}{4} \times (-4) = -1$$

$$g_1 = \frac{1}{4} [0 + 4(0 - 3) + 4(0 - 3) + 4(0 - 3)] = \frac{1}{4} \times (-36) = -9$$

- **Update:**

$$\theta_0 = 0 - 0.1 \times (-1) = \mathbf{0.1}$$

$$\theta_1 = 0 - 0.1 \times (-9) = \mathbf{0.9}$$

- **Step 2:**
 - Compute gradient at $(\theta_0 = 0.1, \theta_1 = 0.9)$:

$$g_0 = \frac{1}{4} [2(0.1 - 2) + 0 + 0 + 0] = \frac{1}{4} \times (-3.8) = -0.95$$

$$g_1 = \frac{1}{4} [0 + 4(0.9 - 3) + 4(0.9 - 3) + 4(0.9 - 3)] = \frac{1}{4} \times (-25.2) = -6.3$$

- **Update:**

$$\theta_0 = 0.1 - 0.1 \times (-0.95) = \mathbf{0.195}$$

$$\theta_1 = 0.9 - 0.1 \times (-6.3) = \mathbf{1.53}$$

Observation: Smooth, direct progress towards $(2, 3)$.

B. Stochastic Gradient Descent (SGD)

Process examples in order: 1, 2, 3, 4. **One step per example.**

- **Step 0:** $\theta_0 = 0, \theta_1 = 0$

- **Step 1 (Example 1):**

- Gradient for Ex. 1: $\mathbf{g} = (2(0 - 2), 0) = (-4, 0)$

- **Update:**

$$\theta_0 = 0 - 0.1 \times (-4) = \mathbf{0.4}$$

$$\theta_1 = 0 - 0.1 \times (0) = \mathbf{0.0}$$

- **Step 2 (Example 2):**

- Gradient for Ex. 2: $\mathbf{g} = (0, 4(0 - 3)) = (0, -12)$

- **Update:**

$$\theta_0 = 0.4 - 0.1 \times (0) = \mathbf{0.4}$$

$$\theta_1 = 0.0 - 0.1 \times (-12) = \mathbf{1.2}$$

Observation: The path is very noisy (θ_0 only updated once, θ_1 updated chaotically), but we've made good overall progress after two steps.

C. Mini-Batch Gradient Descent (Batch size $b = 2$)

Use two mini-batches: Batch A (Examples 1 & 2), Batch B (Examples 3 & 4).

- **Step 0:** $\theta_0 = 0, \theta_1 = 0$

- **Step 1 (Mini-Batch A):**

- Gradient for Batch A (Ex. 1 & 2):

$$g_0 = \frac{1}{2} [2(0 - 2) + 0] = \frac{1}{2} \times (-4) = -2$$

$$g_1 = \frac{1}{2} [0 + 4(0 - 3)] = \frac{1}{2} \times (-12) = -6$$

- **Update:**

$$\theta_0 = 0 - 0.1 \times (-2) = \mathbf{0.2}$$

$$\theta_1 = 0 - 0.1 \times (-6) = \mathbf{0.6}$$

- **Step 2 (Mini-Batch B):**

- Gradient for Batch B (Ex. 3 & 4) at $(\theta_0 = 0.2, \theta_1 = 0.6)$:

$$g_0 = \frac{1}{2} [0 + 0] = 0$$

$$g_1 = \frac{1}{2} [4(0.6 - 3) + 4(0.6 - 3)] = \frac{1}{2} \times (-19.2) = -9.6$$

- **Update:**

$$\theta_0 = 0.2 - 0.1 \times (0) = \mathbf{0.2}$$

$$\theta_1 = 0.6 - 0.1 \times (-9.6) = \mathbf{1.56}$$

Observation: A compromise between the smoothness of Batch GD and the speed of SGD.

D. Newton's Method

- **Step 0:** $\theta_0 = 0, \theta_1 = 0$

- **Step 1:**

- Gradient $\mathbf{g}$: $(2(0 - 2), 4(0 - 3)) = (-4, -12)$

- Update vector: $\mathbf{H}^{-1}\mathbf{g} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{4} \end{bmatrix} \begin{bmatrix} -4 \\ -12 \end{bmatrix} = \begin{bmatrix} -2 \\ -3 \end{bmatrix}$

- **Update:**

$$\theta_0 = 0 - (-2) = \mathbf{2}$$

$$\theta_1 = 0 - (-3) = \mathbf{3}$$

- **Step 2:**

- Gradient at $(2, 3)$: $(2(2 - 2), 4(3 - 3)) = (0, 0)$

- **Update:** $(0, 0)$. **We are already at the minimum!**

Observation: Newton's Method converged in **a single step** for this quadratic function.

Summary Table

Method	Gradient Calculation	Update Step	Pros	Cons
Batch GD	Entire Dataset	$\boldsymbol{\theta} - \eta \mathbf{g}$	Stable, Direct	Very Slow
SGD	Single Example	$\boldsymbol{\theta} - \eta \mathbf{g}^{(i)}$	Very Fast	Chaotic Path
Mini-Batch	Small Subset	$\boldsymbol{\theta} - \eta \mathbf{g}_B$	Fast & Stable	Needs tuning
Newton's	Entire Dataset	$\boldsymbol{\theta} - \mathbf{H}^{-1} \mathbf{g}$	Very Fast Conv.	Extremely Expensive

Key Takeaway: There is a trade-off between the **computational cost per step** and the **number of steps required** to converge. Mini-batch SGD strikes the best practical balance for large-scale learning.